VRHealth Challenge 13 - scripts

Reference documentation of the tools in `VRHealth_Challenge13/scripts`. They convert the Intel RealSense D455 captures (ROS 2 bags recorded with two cameras, `cam1` and `cam2`) for NVIDIA cuVSLAM, run cuVSLAM without ROS, visualize the runs in Rerun, and compare the results with the Isaac ROS runs and with the ground truth.

None of the tools needs ROS: the bags (`.db3` sqlite3 or `.mcap`) are read directly and the ROS 2 messages are decoded by `bag2edex.py`.

Contents

1. Overview
2. Setup
3. Docker
4. Typical workflows
5. Script reference
    - bag2edex.py
    - bag2edex_modes.py
    - convert_all.py
    - merge_rig_edex.py
    - track_edex_rerun.py
    - track_bag_rerun.py
    - track_edex_modes.py
    - bag_output_rerun.py
    - view_frames.py (common 3D frame of the viewers)
    - check_depth_sync.py
    - check_depth_alignment.py
    - run_output.py
    - compare_runs.py
    - db3_to_mcap.sh
    - all_db3_to_mcap.sh
6. File formats
7. Conventions and notes

Overview

Script	Type	What it does
`bag2edex.py`	Python	Converts a ROS 2 bag into one EDEX dataset per camera (RGB, infra1, infra2, IMU), with synchronization, calibration and a quality report of the bag
`bag2edex_modes.py`	Python	Converts a capture into one EDEX per cuVSLAM input configuration (RGB-D, stereo + IMU on each camera, Multicamera, Multisensor), with images shared between the configurations
`convert_all.py`	Python	Runs `bag2edex.py` (or another converter, `--converter`) on every capture of a folder, preserving the folder structure and using the `converted/` copies when present
`merge_rig_edex.py`	Python	Merges the cam1 and cam2 EDEX datasets into a single multi-camera rig, using the extrinsic between the two cameras (default: from the CAD of the mount)
`track_edex_rerun.py`	Python	Runs cuVSLAM (PyCuVSLAM) on an EDEX dataset, shows the run in Rerun and optionally saves the output
`track_bag_rerun.py`	Python	Runs cuVSLAM directly on a ROS 2 bag, without creating an EDEX and without writing images to disk; shows the run in Rerun and optionally saves the output

Script	Type	What it does
`track_edex_modes.py`	Python	Runs cuVSLAM without viewer on an EDEX of `bag2edex_modes.py`, in any odometry mode (also RGBD and Multisensor), and saves the output
`bag_output_rerun.py`	Python	Shows in Rerun a cuVSLAM run already done with Isaac ROS, from the bag of its output topics (and optionally the input bag for the images)
`view_frames.py`	Python module	Common 3D frame of the Rerun viewers (base frame at the first pose) and overlay of saved trajectories (`--world`, `--compare`)
`check_depth_sync.py`	Python	Measures the stamps of native and aligned depth against RGB and IR in every capture
`check_depth_alignment.py`	Python	Compares the aligned depth computed by `bag2edex.py` with the one recorded by the realsense node
`run_output.py`	Python module	Saves the output of a PyCuVSLAM run in the CSV format of the Isaac ROS runs; used by the two tracking scripts (`--out-dir`)
`compare_runs.py`	Python	Compares saved runs with each other and with the ground truth: metrics, tables, plots
`db3_to_mcap.sh`	Bash	Reduces a bag to the topics needed by cuVSLAM and converts it to compressed MCAP
`all_db3_to_mcap.sh`	Bash	Runs `db3_to_mcap.sh` on every capture of a folder, preserving the folder structure
`docker/Dockerfile`	Docker	Image with cuVSLAM, its Python bindings and all the dependencies of the scripts
`docker/run.sh`	Bash	Starts the container with GPU, display, host user and the data folder mounted

Dependencies between the scripts (they must stay in the same folder):

- `track_bag_rerun.py` imports from `bag2edex.py`, `merge_rig_edex.py`, `track_edex_rerun.py` and `run_output.py`;
- `track_edex_rerun.py` imports `run_output.py` (and `bag2edex.py` to read `/tf_static` when `--out-dir` is used);
- `merge_rig_edex.py` and `bag_output_rerun.py` import from `bag2edex.py`;
- the three Rerun viewers (`track_edex_rerun.py`, `track_bag_rerun.py`, `bag_output_rerun.py`) import `view_frames.py`;
- `bag2edex_modes.py` imports from `bag2edex.py` and `merge_rig_edex.py`; `track_edex_modes.py` imports from `track_edex_rerun.py` and `run_output.py`;
- `convert_all.py` runs `bag2edex.py` or the script given with `--converter`; `all_db3_to_mcap.sh` runs `db3_to_mcap.sh`, which runs `bag2edex.py --dry-run` as a check.

Setup

```
cd ~/dotlumen/VRHealth_Challenge13/scripts
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt          # numpy, opencv-python-headless, zstandard, lz4, rerun-sdk
```

Additional requirements:

Needed by	Requirement
`track_edex_rerun.py`, `track_bag_rerun.py`	cuVSLAM Python bindings (built from the cuVSLAM repository) and `rerun-sdk==0.33.1`
`bag_output_rerun.py`	`rerun-sdk==0.33.1` (no cuVSLAM)

Needed by	Requirement
`compare_runs.py`	`matplotlib`; `IcpAlign.py` from `results/ground_truth` for the ground-truth comparison
`db3_to_mcap.sh`, `all_db3_to_mcap.sh`	`rosbags` (`pip install rosbags`), which provides `rosbags-convert`

Docker

`docker/Dockerfile` builds an image that runs every script: it compiles cuVSLAM (default: commit `2a0882c`, version 17.0.0, with cuNLS and LMDB) and its Python bindings on CUDA 12.6 / Ubuntu 24.04, and installs the Python dependencies, `rerun-sdk==0.33.1`, `matplotlib` and `rosbags`. The scripts are copied to `/opt/vrhealth/scripts`, which is on the `PATH`.

Host requirements: NVIDIA driver supporting CUDA 12.6 (version 560 or later), Docker and the NVIDIA Container Toolkit (`--gpus all`).

```
cd ~/dotlumen/VRHealth_Challenge13/scripts
docker build -f docker/Dockerfile -t vrhealth-cuvslam .
# faster: compile only for your GPU (compute capability, e.g. 86 = RTX 30xx, 89 = RTX 40xx)
docker build -f docker/Dockerfile --build-arg CUDA_ARCHITECTURES=89 -t vrhealth-cuvslam .
```

Build argument	Default	Description
`CUVSLAM_REF`	`2a0882c`	cuVSLAM branch, tag or commit to build
`CUVSLAM_REPO`	`https://github.com/nvidia-isaac/cuVSLAM.git`	cuVSLAM repository
`CUDA_ARCHITECTURES`	`all`	CUDA architectures to compile for; `all` works on any GPU but is slow to build
`CUDA_VERSION`, `UBUNTU_VERSION`	`12.6.3`, `24.04`	base images `nvidia/cuda:<cuda>-{devel,runtime}-ubuntu<ubuntu>`

`docker/run.sh` starts the container with the GPU, the host display for the Rerun viewer (X11), the host user id (files written by the scripts belong to you), the data folder mounted at the same absolute path as on the host (so the paths saved in `bag_report.json` and `run_info.json` stay valid) and the scripts folder mounted over `/opt/vrhealth/scripts` (the current version of the scripts is used without rebuilding).

```
docker/run.sh                                              # interactive shell
docker/run.sh track_bag_rerun.py ~/dotlumen/Captures/9_Loop_closure/loop_closure \
    --mode inertial --streams cam2/ir1 cam2/ir2 --rectified --slam --out-dir ~/dotlumen/runs/lc --no-
viewer
DATA=/mnt/data NO_X11=1 docker/run.sh convert_all.py /mnt/data/Captures -o /mnt/data/edex
```

Variable	Default	Description
`IMAGE`	`vrhealth-cuvslam`	image name
`DATA`	`$HOME/dotlumen`	data folder mounted at the same path
`NO_X11`	`0`	`1` = do not share the display (use `--save` or `--no-viewer` in the tracking scripts)
`EXTRA_ARGS`	-	additional `docker run` arguments, e.g. `"-v /mnt/disk:/mnt/disk"`

If the Rerun viewer cannot open a window, save the recording with `--save run.rrd` and open it on the host with `rerun run.rrd`.

When the scripts open the viewer inside the container they stay alive after the run, because the viewer is closed together with the container: close the viewer window (or press Ctrl+C) to end them.

Typical workflows

Convert all captures to EDEX

```
python3 convert_all.py ~/dotlumen/Captures -o ~/dotlumen/Captures/edex
```

Convert all captures to one EDEX per cuVSLAM configuration

```
python3 convert_all.py ~/dotlumen/Captures/original -o ~/dotlumen/Captures/edex --converter
bag2edex_modes.py
```

Reduce the bags (keep only what cuVSLAM needs, compressed MCAP)

```
./all_db3_to_mcap.sh -o ~/dotlumen/Captures/mcap ~/dotlumen/Captures
```

Stereo IR + IMU on one capture, from the EDEX and from the bag, with saved output

```
B=~/dotlumen/Captures/9_Loop_closure/loop_closure
E=~/dotlumen/Captures/edex_ir/9_Loop_closure/loop_closure
R=~/dotlumen/VRHealth_Challenge13/results/pycuvslam

python3 bag2edex.py $B -o $E --reference ir --cameras cam2
python3 track_edex_rerun.py $E/cam2/stereo.edex --mode inertial --cameras 0 1 --rectified --slam \
    --out-dir $R/edex/cam2_stereo_imu/loop_closure --save $R/edex/cam2_stereo_imu/loop_closure/run.rrd
python3 track_bag_rerun.py $B --mode inertial --streams cam2/ir1 cam2/ir2 --rectified --slam \
    --out-dir $R/bag/cam2_stereo_imu/loop_closure --save $R/bag/cam2_stereo_imu/loop_closure/run.rrd
```

Compare with the Isaac ROS run and the ground truth

```
G=~/dotlumen/VRHealth_Challenge13/results
python3 compare_runs.py ros=$G/slam_csv/cam2_stereo_imu/loop_closure \
    bag=$R/bag/cam2_stereo_imu/loop_closure edex=$R/edex/cam2_stereo_imu/loop_closure \
    --gt $G/ground_truth/loop_closure/paths3d.csv --out $G/comparisons/cam2_stereo_imu/loop_closure
```

4-IR rig with both cameras

```
python3 merge_rig_edex.py ~/dotlumen/Captures/edex/9_Loop_closure/loop_closure --streams ir
python3 track_edex_rerun.py ~/dotlumen/Captures/edex/9_Loop_closure/loop_closure/rig_ir/stereo.edex --
mode multicamera
# or directly from the bag
python3 track_bag_rerun.py $B --mode multicamera --streams cam1/ir1 cam1/ir2 cam2/ir1 cam2/ir2
```

Script reference

bag2edex.py

Converts a ROS 2 bag recorded with RealSense cameras into the EDEX format read by cuVSLAM, one dataset per camera. It reads the bag directly (sqlite3 `.db3` or MCAP, also zstd/lz4 compressed) and decodes the `Image`, `CameraInfo`, `Imu` and `TFMessage` messages itself.

What it does

1. Reads the image timestamps, the `camera_info` messages, `/tf_static` and the IMU of the chosen camera.
2. Drops messages whose header stamp is anomalous compared to the bag receive time (`--stale-threshold`), e.g. a frame left queued in the driver when the recording started.
3. Pairs infra1 and infra2 (`--stereo-tolerance-ms`) and matches the IR pairs to the RGB frames (`--tolerance-ms`, 1:1, smallest offsets first).
4. Chooses the reference stream, which defines the EDEX frames and becomes camera 0 (`--reference`): with `rgb` every RGB image is a frame and IR pairs are attached when synchronized; with `ir` every IR pair is a frame and RGB images are attached; `auto` picks the more frequent stream.
5. Writes the calibration: RGB as `brown5k` (ROS `plumb_bob` coefficients reordered), IR as `pinhole` (rectified images). The infra2 baseline is taken from `P[3] = -fx * baseline` of its `camera_info` when it uses the infra1 frame id (realsense-ros convention). Extrinsics come from `/tf_static`; the IMU frame falls back to `<cam>_gyro_optical_frame` when the IMU frame id is not in `/tf_static`.
6. Writes the images (in parallel) and a quality report (brightness, saturated and black pixels, sharpness).

Data kept and dropped

Data	In the EDEX
RGB images (`color/image_raw`)	JPEG (`--jpg-quality`, default 95): all of them with `--reference rgb`, only those within `--tolerance-ms` of an IR pair with `--reference ir`
IR images (`infra1`, `infra2`)	lossless PNG (8-bit): only complete pairs (infra2 within `--stereo-tolerance-ms` of infra1); all pairs with `--reference ir`, only those matched to an RGB frame with `--reference rgb`
Timestamps	header stamp of each image (or receive time with `--time receive`), in `frame_metadata.jsonl` and `sync.csv`
Calibration	intrinsics and distortion from the first `camera_info` of each stream, extrinsics from `/tf_static`, stereo baseline from `P[3]` of infra2
IMU (`<cam>/imu`)	every sample (angular velocity, linear acceleration, timestamp) from 0.5 s before the first frame to 0.5 s after the last, only for `--imu-camera` (default cam2); orientation and covariances are not used
Depth (with `--depth`)	depth aligned to the RGB image, 16-bit PNG in raw units (mm), attached to the RGB frame with the same stamp within `--depth-tolerance-ms` (1 ms): the recorded `aligned_depth_to_color` when the bag has it, otherwise computed from `depth/image_rect_raw`
Not converted	native depth (only used to compute the aligned one), `aligned_depth_to_infra1`, point clouds, separate `accel/gyro` topics, dynamic `/tf`, any other topic; messages with an anomalous stamp (`--stale-threshold`) and duplicated stamps

Depth (`--depth`)

Only the depth aligned to the RGB image is written (`images/depth/NNNNNN.png`, same index as the RGB frame).

- Source (`--depth-source auto`): `/<cam>/aligned_depth_to_color/image_raw` when the bag has messages on it; otherwise it is computed from `/<cam>/depth/image_rect_raw` and its `camera_info` as the librealsense `align` block does: each valid depth pixel is deprojected at its two corners, moved to the RGB frame with the `/tf_static` extrinsic, projected with the RGB intrinsics and distortion (`--depth-color-model`), and its footprint is filled with the raw depth value, keeping the nearest value when several pixels overlap. `recorded` or `computed` force one source.
- Synchronization: cuVSLAM has no timestamp for depth and assumes it was taken with the image, so each depth image is attached to the RGB frame with the same stamp within `--depth-tolerance-ms` (default 1 ms). The recorded aligned depth carries the stamp of the native depth

frame, which coincides with an RGB frame when both streams are hardware-synced; depth runs at ~15 Hz against ~30 Hz RGB, so about half of the RGB frames get a depth (nearly all with `--reference ir`). The real depth stamp is written to `frame_metadata.jsonl`; `sync.csv` and `bag_report.json` give the RGB-depth offset, the frames with depth and the share of valid pixels.
- EDEX fields: the RGB camera gets `depth_id: 0` and `depth_scale_factor` (`--depth-scale-factor`, 1000 = mm), the body a `depth_sequence`, each frame with depth a `depth` list (`id`, `filename`, `timestamp`). With `--reference rgb` this is the layout of the cuVSLAM RGB-D datasets. With `--reference ir` the RGB is camera 2: the cuVSLAM C++ EDEX reader uses `depth_id` as the index of the depth stream, the cuVSLAM Python tools as a camera id.
- `check_depth_sync.py` measures the depth stamps of all captures; `check_depth_alignment.py` compares the computed alignment with the recorded one on bags that have both.

Camera order in the EDEX: `rgb, ir1, ir2` with `--reference rgb`; `ir1, ir2, rgb` with `--reference ir`. Use these indices in `track_edex_rerun.py --cameras`.

Output

```
<output>/
├── bag_report.json           topic statistics, gaps, image quality, calibration, dropped messages
└── <cam>/
    ├── stereo.edex           EDEX header and body (cameras, extrinsics, IMU section)
    ├── frame_metadata.jsonl  one line per frame: image file and timestamp of each camera present
    ├── calibration.json      original camera_info, transforms used, IMU frame
    ├── sync.csv              per frame: stamps of RGB / infra1 / infra2, receive time, RGB-IR offset
    ├── IMU.jsonl, imu.csv    IMU samples (only for --imu-camera)
    └── images/{rgb,infra1,infra2}/000000.{jpg,png}
```

Usage

```
python3 bag2edex.py <bag> -o <output> [options]
python3 bag2edex.py ~/dotlumen/Captures/9_Loop_closure/loop_closure -o /tmp/lc --dry-run   # analysis only
```

Arguments and options

Argument	Value	Default	Description
`bag` (positional)	text	**required**	bag folder (containing the .db3/.mcap files) or a single file
`-o`, `--output`	text	**required**	output folder
`--cameras`	one or more `CAMERAS`	-	camera prefixes (default: detected from /<cam>/color/image_raw)
`--rgb-topic`	text	`/{cam}/color/image_raw`	RGB image topic; `{cam}` is replaced by the camera prefix
`--rgb-info-topic`	text	`/{cam}/color/camera_info`	RGB camera_info topic
`--ir1-topic`	text	`/{cam}/infra1/image_rect_raw`	infra1 image topic
`--ir1-info-topic`	text	`/{cam}/infra1/camera_info`	infra1 camera_info topic
`--ir2-topic`	text	`/{cam}/infra2/image_rect_raw`	infra2 image topic

Argument	Value	Default	Description
`--ir2-info-topic`	text	`/{cam}/infra2/camera_info`	infra2 camera_info topic (its `P[3]` gives the stereo baseline)
`--imu-camera`	text	`cam2`	camera that owns the IMU ('none' to skip exporting it)
`--imu-topic`	text	`/{cam}/imu`	IMU topic (`sensor_msgs/Imu`, gyroscope and accelerometer combined)
`--time`	`header` | `receive`	`header`	time used for synchronization and written timestamps
`--tolerance-ms`	number	`10.0`	maximum RGB <-> IR pair offset
`--stereo-tolerance-ms`	number	`1.0`	maximum infra1 <-> infra2 offset
`--start`	number	-	seconds from the start of the bag at which to start
`--end`	number	-	seconds from the start of the bag at which to stop
`--jpg-quality`	integer	`95`	JPEG quality of the RGB images (0-100)
`--png-compression`	`0` | `1` | `2` | `3` | `4` | `5` | `6` | `7` | `8` | `9`	`6`	PNG compression level (lossless; 9 = smallest files)
`--ir-format`	`png` | `jpg`	`png`	image format of infra1/infra2 (`png` is lossless)
`--gyro-noise-density`	number	`0.000244`	gyroscope noise density written to the EDEX IMU section [rad/s/sqrt(Hz)]
`--gyro-random-walk`	number	`1.9393e-05`	gyroscope random walk written to the EDEX [rad/s^2/sqrt(Hz)]
`--accel-noise-density`	number	`0.001862`	accelerometer noise density written to the EDEX [m/s^2/sqrt(Hz)]
`--accel-random-walk`	number	`0.003`	accelerometer random walk written to the EDEX [m/s^3/sqrt(Hz)]
`--imu-frequency`	number	-	IMU frequency for the EDEX (default: estimated from the data)
`--workers`	integer	number of CPUs, between 2 and 8	threads for image encoding
`--overwrite`	flag	off	overwrite the output folder
`--stale-threshold`	number	`0.5`	drop messages whose stamp-receive offset deviates from the median by more than N seconds (0 = disabled)
`--reference`	`rgb` | `ir` | `auto`	`rgb`	stream that defines the frames (camera 0): rgb, ir pair, or auto = the most frequent
`--dry-run`	flag	off	analyze and synchronize without writing images
`--depth`	flag	off	also write the depth aligned to the RGB image (images/depth, 16-bit PNG)
`--depth-source`	`auto` | `recorded` | `computed`	`auto`	auto = recorded aligned depth when present, otherwise computed from the native depth
`--depth-tolerance-ms`	number	`1.0`	maximum RGB <-> depth offset (cuVSLAM assumes depth taken with the image)

Argument	Value	Default	Description
`--aligned-depth-topic`	text	`/{cam}/aligned_depth_to_color/image_raw`	
`--depth-topic`	text	`/{cam}/depth/image_rect_raw`	native depth, used to compute the aligned depth
`--depth-info-topic`	text	`/{cam}/depth/camera_info`	
`--depth-scale-factor`	number	`1000.0`	depth units per meter, written to the EDEX (RealSense: 1000 = millimeters)
`--depth-color-model`	`plumb_bob` | `inverse`	`plumb_bob`	distortion model used to project into the RGB image when computing the aligned depth

bag2edex_modes.py

Converts one capture (`cam1` and `cam2`) into one EDEX per cuVSLAM input configuration. The images are written once in the capture folder and shared by all the configurations. It reads the original bags, because the reduced MCAP bags do not contain the native depth. Use `bag2edex.py` for per-camera datasets with the full quality report.

Configurations (a configuration is skipped, with the reason in `modes.json`, when a stream is missing)

Folder	Cameras (in EDEX order)	Depth	IMU	cuVSLAM modes
`rgbd_ir_<cam>`	infra1	native depth (camera 0)	-	RGBD
`rgbd_rgb_<cam>`	RGB (`brown5k`)	aligned to the RGB (camera 0)	-	RGBD
`stereo_imu_<cam>`	infra1, infra2	-	same camera	Inertial, stereo (Multicamera), Mono (`--cameras 0`)
`multicamera_ir`	cam2 infra1, cam2 infra2, cam1 infra1, cam1 infra2	-	-	Multicamera, experimental
`multisensor_ir_depth`	cam2 infra1, cam1 infra1, cam2 infra2, cam1 infra2	native depth on the two infra1	cam2	Multisensor, experimental
`multisensor_all`	as above + cam2 RGB, cam1 RGB rectified to pinhole	native depth on the two infra1	cam2	Multisensor, experimental

Captures without infra2 keep only the configurations that do not need it, and the two Multisensor configurations use infra1 only.

Depth: the native depth has the same stamps, intrinsics and viewpoint as infra1, so it is pixel-aligned with it. RGB-D needs a depth in every frame, so only the frames with a depth are kept (within `--depth-tolerance-ms`). The depth aligned to the RGB is the recorded one when the bag has it; otherwise it is computed from the native depth as in `bag2edex.py`, within `--computed-depth-tolerance-ms`. Cameras with depth come first in the EDEX, because `depth_id` must match both the camera index and the `depth_sequence` index; for this reason the Multisensor configurations cannot use `rectified_stereo_camera`.

cam1 + cam2 configurations (experimental): the two D455 are not hardware-synchronized. The frames are defined by the stereo pairs of the reference camera (`--reference-cam`, default cam2). The cam1 pair and the RGB images within `--rig-tolerance-ms` are attached and all the images of a frame get the timestamp of the reference camera; frames without cam1 stay partial, cam1 frames without a match are discarded. The real stamps are in `true_timestamp` and `sync.csv`. These images are not

simultaneous: at 200 deg/s, 15 ms correspond to about 20 pixels between the cameras. cam1 is connected to cam2 with the CAD extrinsic (`--extrinsic`).

RGB rectification (`rgb_rect`, Multisensor only supports pinhole cameras): for each pixel `(u', v')` of the rectified image, with `K` and `D = [k1, k2, p1, p2, k3]` of `camera_info`,

```
x = (u' - cx) / fx        y = (v' - cy) / fy        r^2 = x^2 + y^2
c = 1 + k1 r^2 + k2 r^4 + k3 r^6
xd = x c + 2 p1 x y + p2 (r^2 + 2 x^2)
yd = y c + 2 p2 x y + p1 (r^2 + 2 y^2)
u = fx xd + cx            v = fy yd + cy
```

and the source image is sampled at `(u, v)` with bicubic interpolation (replicated border). `R` is the identity, so frame and extrinsics do not change, and the new camera matrix is `K`, so the resolution stays 640x480. On the D455 captures no pixel falls outside the source and the shift is at most about 6 pixels. `--rgb-distortion-scale` multiplies all the coefficients: a comparison of the infra1 and RGB gradients suggests that 0.4-0.7 fits better than the factory values, but the default is 1 until a calibration is available.

Output

```
<output>/
├── images/<cam>/{infra1,infra2,rgb,rgb_rect,depth_native,depth_aligned}/NNNNNN.{png,jpg}
├── imu/IMU_<cam>.jsonl
├── bag_report.json          path of the source bag
├── modes.json               per configuration: frames, cameras, offsets, predicted stereo pairs,
skipped ones
└── <configuration>/
    ├── stereo.edex
    ├── frame_metadata.jsonl
    ├── sync.csv             per frame: real stamp of each camera and offset from the frame stamp
    └── calibration.json     rig frame, T_rig_from_base_opencv, cameras, IMU, rectification
```

No links are used: paths inside a configuration are relative (`../images/...`, `../imu/...`), so a configuration needs the whole capture folder. `sequence` and `depth_sequence` have one entry per frame (empty for a missing image).

Usage

```
python3 bag2edex_modes.py ~/dotlumen/Captures/original/9_Loop_closure/loop_closure -o
~/dotlumen/Captures/edex/9_Loop_closure/loop_closure
python3 bag2edex_modes.py <capture> -o <output> --modes stereo_imu multisensor_all --dry-run
```

Arguments and options

Argument	Value	Default	Description
`bag` (positional)	text	**required**	capture folder (.db3/.mcap)
`-o`, `--output`	text	**required**	output folder of this capture
`--modes`	one or more of `rgbd_ir`, `rgbd_rgb`, `stereo_imu`, `multicamera_ir`, `multisensor_ir_depth`, `multisensor_all`	all	configurations to write
`--stereo-tolerance-ms`	number	`1.0`	maximum infra1 <-> infra2 offset

Argument	Value	Default	Description
`--depth-tolerance-ms`	number	`1.0`	maximum image <-> depth offset (infra1 + native depth, RGB + recorded aligned depth)
`--computed-depth-tolerance-ms`	number	`8.0`	maximum RGB <-> native depth offset when the aligned depth is computed
`--rig-tolerance-ms`	number	`16.0`	maximum offset of cam1 and RGB images from the reference frame in the cam1 + cam2 configurations
`--reference-cam`	text	`cam2`	camera defining the frames of the cam1 + cam2 configurations
`--extrinsic`	`X` `Y` `Z` `QX` `QY` `QZ` `QW`	`-0.04 0.0 0.0509 0.0 -0.42273 0.0 0.90626`	T_cam1_link_from_cam2_link (default: CAD)
`--depth-scale-factor`	number	`1000.0`	depth units per meter
`--depth-color-model`	`plumb_bob` | `inverse`	`plumb_bob`	RGB distortion model when computing the aligned depth
`--rgb-distortion-scale`	number	`1.0`	factor applied to the `camera_info` distortion coefficients when rectifying the RGB
`--stale-threshold`	number	`0.5`	drop messages whose stamp-receive offset is more than N s from the median (0 = disabled)
`--gyro-noise-density`, `--gyro-random-walk`	number	`0.000244`, `1.9393e-05`	gyroscope noise written to the EDEX
`--accel-noise-density`, `--accel-random-walk`	number	`0.001862`, `0.003`	accelerometer noise written to the EDEX
`--jpg-quality`	integer	`95`	JPEG quality of the RGB images
`--png-compression`	`0-9`	`6`	PNG compression of IR and depth
`--workers`	integer	number of CPUs, between 2 and 8	threads for image encoding
`--overwrite`	flag	off	replace the output folder (a `convert.log` in it is kept)
`--dry-run`	flag	off	build the configurations without writing anything

convert_all.py

Converts every capture of a folder to EDEX by running `bag2edex.py --reference auto --overwrite` on each one, and mirrors the folder structure in the output folder.

Rules

- A capture is a folder that directly contains `.db3` or `.mcap` files.

- If a capture has a `converted/` subfolder with a bag, that copy is used; the output still goes to the capture path (without `converted`).
- Folders whose name starts with `vslam`, `converted` folders and the output folder are skipped.
- Bags without camera topics (e.g. SLAM output only) are reported with status `no cameras`.

Output: for each capture, the `bag2edex.py` output and `convert.log`; at the end `<output>/conversion_summary.json` (source, status, return code, duration, last log lines of each capture).

Usage

```
python3 convert_all.py ~/dotlumen/Captures -o ~/dotlumen/Captures/edex --list     # only list the
captures
python3 convert_all.py ~/dotlumen/Captures -o ~/dotlumen/Captures/edex
python3 convert_all.py ~/dotlumen/Captures -o ~/dotlumen/Captures/edex --only
9_Loop_closure/loop_closure \
    --extra "--stale-threshold 1.0"
```

Arguments and options

Argument	Value	Default	Description
`root` (positional)	text	**required**	captures folder (e.g. ~/dotlumen/Captures)
`-o`, `--output`	text	**required**	output folder (e.g. ~/dotlumen/Captures/edex)
`--list`	flag	off	list the captures found and exit
`--skip-existing`	flag	off	skip captures that already have bag_report.json in the output
`--only`	one or more `ONLY`	-	convert only these relative paths
`--workers`	integer	number of CPUs, between 2 and 8	threads for image encoding, passed to `bag2edex.py`
`--converter`	text	`bag2edex.py`	conversion script, in the folder of `convert_all.py`; `--reference auto` is passed only to `bag2edex.py`
`--extra`	text	``	extra arguments for the conversion script

merge_rig_edex.py

Merges the per-camera EDEX datasets of a capture (`cam1/` and `cam2/`, produced by `bag2edex.py`) into a single EDEX rig, so that cuVSLAM can use both cameras together. The per-camera folders are not modified.

What it does

1. Reads `stereo.edex`, `calibration.json` and `frame_metadata.jsonl` of both cameras.
2. Reads `/tf_static` of the source bag (path from `bag_report.json`, or `--bag`) to get `<cam>_link -> optical frame` of each camera, and connects the two cameras with `--extrinsic` (`T_<cam1>_link_from_<cam2>_link`). The default is the transform measured on the CAD of the mount: 50 degrees of relative pitch, cam2 about 5 cm above and 4 cm behind cam1.
3. Builds the frames on the first stream of the first camera and matches the other streams by timestamp (`--tolerance-ms`, cuVSLAM requires <= 1 ms); frames without a match stay partial.
4. Runs the same frustum-overlap test used by cuVSLAM to form stereo pairs (pairs above 0.5) and warns when no pair is formed: the cam1/cam2 RGB pair overlaps about 20% and is not accepted by cuVSLAM.

Output (`<capture>/rig_<streams>/` by default): `stereo.edex`, `frame_metadata.jsonl`, `sync.csv`, `rig_calibration.json`, `IMU.jsonl` (link to the IMU of the camera that has it) and `images/<cam>` (links, or copies with `--copy`).

Notes: with more than two cameras cuVSLAM uses the IMU only to estimate gravity (no visual-inertial fusion). The CAD extrinsic is accurate to a few millimetres; a Kalibr calibration is recommended.

Usage

```
python3 merge_rig_edex.py ~/dotlumen/Captures/edex/9_Loop_closure/loop_closure --streams ir
python3 merge_rig_edex.py <capture> --streams all --extrinsic -0.040 0 0.051 0 -0.4227 0 0.9063 --
tolerance-ms 3
```

Arguments and options

Argument	Value	Default	Description
`capture` (positional)	text	**required**	folder containing cam1/ and cam2/ (output of bag2edex.py)
`-o`, `--output`	text	-	output folder (default <capture>/rig_<streams>)
`--cams`	2 values	`cam1 cam2`	the two cameras; the first one is the reference
`--streams`	`ir` | `rgb` | `all`	`ir`	streams of each camera put in the rig: `ir` (infra1 + infra2), `rgb`, `all` (IR first, then RGB)
`--extrinsic`	`X` `Y` `Z` `QX` `QY` `QZ` `QW`	`-0.04 0.0 0.0509 0.0 -0.42273 0.0 0.90626`	T_<cam1>*link_from*<cam2>_link in meters and quaternion (default: CAD)
`--link-frame`	text	`{cam}_link`	name of each camera's base frame
`--bag`	text	-	bag to read /tf_static from (default: from bag_report.json)
`--tolerance-ms`	number	`1.0`	maximum offset between the reference and the other streams
`--copy`	flag	off	copy images instead of using symbolic links
`--overwrite`	flag	off	replace the output folder if it exists

track_edex_rerun.py

Runs cuVSLAM through its Python bindings (PyCuVSLAM) on an EDEX dataset with `frame_metadata.jsonl`, and visualizes the run in Rerun.

What it does

1. Loads the cameras selected with `--cameras` and re-centres the rig on the first of them (required by Mono, and the trajectory is that camera's trajectory). Converts the EDEX extrinsics (legacy cuVSLAM axes) to OpenCV axes.
2. In Inertial mode, feeds the IMU samples up to each frame; samples before the first frame are discarded.
3. Tracks every frame that contains at least one selected camera (Mono: camera 0 must be present). Frames whose cameras are more than 1 ms apart are reported.
4. Logs to Rerun: odometry trajectory, SLAM trajectory (`--slam`), rig pose, frustums, images with the tracked points, landmarks, final point cloud, and IMU plots and gravity vector in Inertial

mode.

5. With `--out-dir`, saves the output through `run_output.py` (see File formats). The poses are converted from the rig to the base frame (`--base-frame`, default `<cam>_link`) using `calibration.json` and `/tf_static` of the source bag.

Usage

```
python3 track_edex_rerun.py <stereo.edex> --mode inertial --cameras 0 1 --rectified --slam \
    --out-dir runs/edex/loop_closure --no-viewer
python3 track_edex_rerun.py <stereo.edex> --mode mono --cameras 0 --save run.rrd --tum odom.txt
```

Arguments and options

Argument	Value	Default	Description
`edex` (positional)	text	**required**	stereo.edex file
`--mode`	`multicamera` | `inertial` | `mono`	`multicamera`	cuVSLAM odometry mode
`--cameras`	one or more `CAMERAS`	-	indices of the EDEX cameras to use (default: mono -> 0, otherwise all)
`--slam`	flag	off	also enable SLAM
`--rectified`	flag	off	rectified_stereo_camera=True (pinhole camera pairs only, e.g. --cameras 1 2)
`--start`	integer	`0`	first frame to process (index)
`--max-frames`	integer	-	maximum number of frames to process
`--log-input`	flag	off	also log the exact array passed to cuVSLAM as a plain 2D image (entity input/camN), to compare it with the frustum visualization
`--image-every`	integer	`1`	send images to Rerun every N frames (makes the recording lighter)
`--save`	text	-	save to an .rrd file instead of opening the viewer
`--tum`	text	-	save the odometry trajectory in TUM format
`--verbosity`	integer	`1`	cuvslam.set_verbosity (0-3)
`--out-dir`	text	-	save the cuVSLAM output (CSV in the Isaac ROS format + run_info.json) to this folder
`--base-frame`	text	-	frame of the saved poses (default: <cam>_link of the EDEX rig; 'rig' = no conversion)
`--bag`	text	-	source bag for /tf_static (default: from ../bag_report.json)
`--async-sba`	flag	off	async_sba=True (as in real time; default False: deterministic offline run)
`--no-viewer`	flag	off	do not open the Rerun viewer

Argument	Value	Default	Description
`--world`	`base` | `native`	`base`	3D world frame: 'base' = base frame (--base-frame) at the first pose, z up, the same for every viewer (see view_frames.py); 'native' = cuVSLAM rig at the first frame (OpenCV axes)
`--compare`	one or more `LABEL=FOLDER`	-	overlay the trajectories saved in these folders (odometry.csv / slam_poses.csv of Isaac ROS or of --out-dir); needs --world base

track_bag_rerun.py

Runs cuVSLAM directly on a ROS 2 bag (`.mcap` or `.db3`) and visualizes the run in Rerun, doing in memory what `bag2edex.py` + `merge_rig_edex.py` + `track_edex_rerun.py` do, without writing images to disk.

What it does

1. Pass 1: reads image timestamps, `camera_info`, `/tf_static` and the IMU; drops stale messages as `bag2edex.py`.
2. Builds the rig from `--streams` (`<cam>/<rgb|ir1|ir2>`). The first stream is the reference: it defines the frames, their timestamps and the rig origin. Streams of two cameras are connected with `--extrinsic` when `/tf_static` does not connect them. The other streams are matched to the reference within `--tolerance-ms`.
3. Pass 2: reads the images in file order and feeds the frames to cuVSLAM in timestamp order.
4. Rerun layout: the image of each camera with its tracked points (top left), gyroscope, accelerometer and number of tracked points (bottom left), 3D view with trajectory, rig and landmarks (right).
5. With `--out-dir`, saves the output through `run_output.py`, with the poses of `<cam>_link` of the reference stream.

Default streams: Mono -> `<imu-camera>/rgb`; Inertial -> `<imu-camera>/ir1 <imu-camera>/ir2`; Multicamera -> `ir1 ir2` of every camera in the bag.

Incomplete frames: frames where some stream has no image within `--tolerance-ms` (e.g. infra1 without infra2) are dropped in inertial mode by default (`--require-all-streams`), as in the EDEX created with `bag2edex.py --reference ir` and in the Isaac ROS node (`min_num_images: 2`); with `--no-require-all-streams`, and by default in the other modes, they are passed to cuVSLAM as partial frames. `run_info.json` records the setting and the number of frames before and after the check. On `loop_closure` (IR stereo + IMU) 826 infra1 frames are found, 804 with infra2: the default inertial run uses those 804.

Usage

```
B=~/dotlumen/Captures/mcap/9_Loop_closure/loop_closure
python3 track_bag_rerun.py $B --mode inertial --streams cam2/ir1 cam2/ir2 --rectified --slam --out-dir runs/bag/lc
python3 track_bag_rerun.py $B --mode multicamera --streams cam1/ir1 cam1/ir2 cam2/ir1 cam2/ir2 --save lc.rrd
python3 track_bag_rerun.py $B --mode mono --streams cam2/rgb --start 10 --end 40
```

Arguments and options

Argument	Value	Default	Description
`bag` (positional)	text	**required**	bag folder (.mcap/.db3) or single file

Argument	Value	Default	Description
`--mode`	`multicamera` | `inertial` | `mono`	`multicamera`	cuVSLAM odometry mode
`--streams`	one or more `STREAMS`	-	streams <cam>/<rgb|ir1|ir2>; the first one is the reference. Default: mono -> <imu-camera>/rgb, inertial -> <imu-camera>/ir1 ir2, multicamera -> ir1 ir2 of all cameras
`--imu-camera`	text	`cam2`	camera providing the IMU (inertial mode)
`--extrinsic`	`X Y Z QX QY QZ QW`	`-0.04 0.0 0.0509 0.0 -0.42273 0.0 0.90626`	T_<cam1>*link_from*<cam2>_link (used only if /tf_static does not connect the cameras)
`--link-frame`	text	`{cam}_link`	base frame of each camera in /tf_static
`--require-all-streams`, `--no-require-all-streams`	flag	on in inertial mode, off otherwise	drop the frames where some stream has no image within --tolerance-ms (as the EDEX created with bag2edex.py and the Isaac ROS node, min_num_images). Default: on in inertial mode, off otherwise (partial frames are passed to cuVSLAM)
`--tolerance-ms`	number	`1.0`	maximum offset between the reference and the other streams (cuVSLAM requires <= 1 ms)
`--stale-threshold`	number	`0.5`	discard messages whose stamp-receive offset is more than N s from the median
`--multicam-mode`	`precision` | `moderate` | `performance`	`precision`	selection of primary cameras in Multicamera
`--slam`	flag	off	also enable SLAM
`--rectified`	flag	off	rectified_stereo_camera=True (rectified IR pairs only)
`--start`	number	-	seconds from the beginning at which to start
`--end`	number	-	seconds from the beginning at which to stop
`--max-frames`	integer	-	maximum number of frames to process
`--image-every`	integer	`1`	send images to Rerun every N frames
`--save`	text	-	save to an .rrd file instead of opening the viewer
`--tum`	text	-	save the odometry trajectory in TUM format
`--slam-tum`	text	-	save the SLAM trajectory in TUM format
`--verbosity`	integer	`1`	cuvslam.set_verbosity (0-3)
`--out-dir`	text	-	save the cuVSLAM output (CSV in the Isaac ROS format + run_info.json) to this folder
`--base-frame`	text	-	frame of the saved poses (default: <cam>_link of the reference stream; 'rig' = no conversion)
`--async-sba`	flag	off	async_sba=True (as in real time; default False: deterministic offline run)
`--no-viewer`	flag	off	do not open the Rerun viewer
`--world`	`base` | `native`	`base`	3D world frame: 'base' = base frame (--base-frame) at the first pose, z up, the same for every viewer (see view_frames.py); 'native' = cuVSLAM rig at the first frame (OpenCV axes)

Argument	Value	Default	Description
`--compare`	one or more `LABEL=FOLDER`	-	overlay the trajectories saved in these folders (odometry.csv / slam_poses.csv of Isaac ROS or of --out-dir); needs --world base

track_edex_modes.py

Runs cuVSLAM (PyCuVSLAM) on one EDEX written by `bag2edex_modes.py`, without viewer, and saves the output through `run_output.py`. It adds the RGBD and Multisensor modes to those of `track_edex_rerun.py`; depth images are read from the `depth` entries of `frame_metadata.jsonl`. The poses are converted with `T_rig_from_base_opencv` of `calibration.json`, so the bag is not needed.

```
E=~/dotlumen/Captures/edex/9_Loop_closure/loop_closure
python3 track_edex_modes.py $E/rgbd_ir_cam2/stereo.edex --mode rgbd --slam --out-dir runs/rgbd_ir_cam2
python3 track_edex_modes.py $E/stereo_imu_cam2/stereo.edex --mode inertial --rectified --slam --out-dir
runs/stereo_imu_cam2
python3 track_edex_modes.py $E/multisensor_all/stereo.edex --mode multisensor --slam --out-dir
runs/multisensor_all
```

Argument	Value	Default	Description
`edex` (positional)	text	**required**	stereo.edex file
`--mode`	`mono` | `multicamera` | `inertial` | `rgbd` | `multisensor`	**required**	cuVSLAM odometry mode
`--cameras`	one or more `CAMERAS`	all	EDEX cameras to use
`--no-imu`	flag	off	Multisensor: ignore the IMU of the EDEX
`--rectified`	flag	off	rectified_stereo_camera=True (pinhole stereo pairs in consecutive positions)
`--slam`	flag	off	also enable SLAM
`--max-frames`	integer	-	maximum number of frames
`--out-dir`	text	**required**	output folder (Isaac ROS CSV format + run_info.json)
`--base-frame`	text	-	frame of the saved poses (default: `base_frame` of `calibration.json`)
`--bag`	text	-	source bag for /tf_static when `calibration.json` has no transform
`--verbosity`	integer	`1`	cuvslam.set_verbosity (0-3)

bag_output_rerun.py

Shows in Rerun a cuVSLAM run already done with Isaac ROS (`isaac_ros_visual_slam`), reading the bag that recorded its output topics (for example `<run>/bag` written by `isaac_ros_5/launch/slam_cam2.sh`). It does not need ROS or cuVSLAM: it only reads the bags and logs to Rerun.

What it does

1. Reads the ROS 2 message definitions stored in the MCAP file and decodes every message type with them, including `isaac_ros_visual_slam_interfaces/VisualSlamStatus`. For `.db3` bags, which do

not store the definitions, built-in definitions of the common types are used (`Odometry`, `Path`, `PointCloud2`, `PoseArray`, `PoseStamped`, `TFMessage`).
2. Places every message on the `time` timeline by its header stamp, relative to the first odometry message.
3. Keeps the latest transforms of `/tf` and `/tf_static` and expresses everything in the world frame (default: the `frame_id` of the odometry, e.g. `odom`); for example `slam_path`, published in `map`, is moved to `odom` with the `map -> odom` transform.
4. Logs (same layout and entity names as `track_bag_rerun.py`):

Entity	Content
`world/odom_trajectory`	odometry trajectory (line)
`world/rig`	current pose of the odometry child frame (e.g. `cam2_link`)
`world/rig/<cam>_<role>`	camera frustum and image, with `--input-bag`
`world/vo_path`, `world/slam_path`	last `nav_msgs/Path` published; `slam_path` is the trajectory after the loop closure corrections
`world/vis/<name>`	point clouds and pose arrays of `/visual_slam/vis/*` (landmarks, observations, loop closures, pose graph)
`stats/status/<field>`	numeric fields of `/visual_slam/status` (`vo_state`, `integrator_state`, ...)
`stats/timing/<field>`	execution times of `/visual_slam/status`
`stats/points/<name>`	number of points of each point cloud topic

5. With `--input-bag`, reads `camera_info` and `/tf_static` of the bag played during the run to place the cameras on the rig and logs their images on the same timeline, so they line up with the output of cuVSLAM.

The 3D view uses the common frame of the viewers (see below): with `--compare` the trajectories of PyCuVSLAM runs saved with `--out-dir` are overlaid on the Isaac ROS run.

Usage

```
R=<slam_runs>/cam2_stereo_imu/loop_closure
python3 bag_output_rerun.py $R/bag
python3 bag_output_rerun.py $R/bag --input-bag ~/dotlumen/Captures/9_Loop_closure/loop_closure \
    --streams cam2/ir1 cam2/ir2 --save $R/isaac_ros.rrd
```

Arguments and options

Argument	Value	Default	Description
`output_bag` (positional)	text	**required**	bag with the cuVSLAM output topics (folder, .mcap or .db3)
`--input-bag`	text	-	bag played during the run, for images and calibration
`--streams`	one or more `STREAMS`	-	images to show, <cam>/<rgb|ir1|ir2> (default with --input-bag: <cam>/ir1, where <cam> is the prefix of the odometry child frame, e.g. cam2_link -> cam2)
`--odometry-topic`	text	`/visual_slam/tracking/odometry`	
`--status-topic`	text	`/visual_slam/status`	

Argument	Value	Default	Description
`--world-frame`	text	-	frame of the 3D view (default: frame_id of the odometry, e.g. odom)
`--image-every`	integer	`1`	log one image every N
`--cloud-every`	integer	`1`	log one message every N for each point cloud topic
`--save`	text	-	save to a .rrd file instead of opening the viewer
`--world`	`base` | `native`	`base`	3D world frame: 'base' = odometry child frame (e.g. cam2_link) at the first pose, the same for every viewer (see view_frames.py); 'native' = world frame of Isaac ROS (e.g. odom)
`--compare`	one or more `LABEL=FOLDER`	-	overlay the trajectories saved in these folders (odometry.csv / slam_poses.csv of Isaac ROS or of --out-dir of the PyCuVSLAM scripts); needs --world base

Common 3D frame of the viewers (view_frames.py)

`track_edex_rerun.py`, `track_bag_rerun.py` and `bag_output_rerun.py` show the 3D view in the same frame (`--world base`, default): the base frame (e.g. `cam2_link`: x forward, y left, z up) at the first pose of the run. It is the convention of the saved CSV files (`run_output.py`, `extract_paths.py`): the first pose is the identity.

Tool	Native world frame (`--world native`)	Conversion to the common frame
PyCuVSLAM (`track_*_rerun.py`)	first camera of the rig at the first frame, OpenCV axes (z forward, y down)	`inv(W_b0) @ W_b`, with `W_b = world_from_rig @ T_rig_from_base` (`T_rig_from_base` from `/tf_static`)
Isaac ROS (`bag_output_rerun.py`)	`odom` (z up, origin and heading set by the node)	`inv(T_odom_base0) @ T_odom_base`

Landmarks, gravity and cameras are moved to the same frame. `--compare label=folder` (repeatable) overlays the trajectories saved in other run folders (`odometry.csv`, `slam_poses.csv` of Isaac ROS or of `--out-dir`), re-anchored at their pose closest to the first timestamp of the current run, so that runs starting a few frames apart start at the same point. Different recordings cannot be overlaid in one Rerun view: `--compare` puts the other runs in the same recording, under `world/compare/<label>/`.

```
python3 bag_output_rerun.py <isaac run>/bag --input-bag <capture> \
    --compare
pycuvslam=~/dotlumen/VRHealth_Challenge13/results/pycuvslam/bag/cam2_stereo_imu/loop_closure
python3 track_bag_rerun.py <capture> --mode inertial --streams cam2/ir1 cam2/ir2 --rectified --slam \
    --compare isaac_ros=~/dotlumen/VRHealth_Challenge13/results/slam_csv/cam2_stereo_imu/loop_closure
```

check_depth_sync.py

Diagnostic tool: for every capture of a folder (same rules as `convert_all.py`) and every camera it reads only the message headers of RGB, infra1, native depth and aligned depth, and reports for each pair the offset of each message from the nearest message of the other topic (share of identical stamps, median, 95th percentile, maximum, share within 1, 2, 5, 8, 10 ms). Pairs: aligned -> rgb, aligned -> depth, rgb -> depth, rgb -> aligned, depth -> ir1.

```
python3 check_depth_sync.py ~/dotlumen/Captures -o ~/dotlumen/Captures/depth_sync.json
```

Arguments and options

Argument	Value	Default	Description
`root` (positional)	text	**required**	captures folder (e.g. ~/dotlumen/Captures)
`-o`, `--output`	text	`depth_sync.json`	JSON file with all the statistics
`--only`	one or more `ONLY`	-	only these relative capture paths

check_depth_alignment.py

Checks the depth alignment computed by `bag2edex.py` (`align_depth_to_color`) against the aligned depth recorded by the realsense node, on bags that have both `aligned_depth_to_color` and `depth/image_rect_raw`. On a sample of frames with the same stamp on both topics it computes the alignment with both RGB distortion models (`plumb_bob` and `inverse`) and reports, as medians over the frames: share of pixels with the same value, share within 1% or 2 cm, median absolute difference, and pixels with depth in only one of the two images. `--save-dir` writes the recorded, computed and difference images.

```
python3 check_depth_alignment.py ~/dotlumen/Captures/9_Loop_closure/loop_closure --cam cam2 --frames 20
-o align_cam2.json
```

Arguments and options

Argument	Value	Default	Description
`bag` (positional)	text	**required**	
`--cam`	text	`cam2`	
`--frames`	integer	`20`	number of frames sampled over the bag
`--depth-scale-factor`	number	`1000.0`	
`--save-dir`	text	-	write recorded / computed / difference images here
`-o`, `--output`	text	-	JSON file with the results

run_output.py

Python module (no command line) used by `track_edex_rerun.py` and `track_bag_rerun.py` when `--out-dir` is given. It collects the per-frame output of the cuVSLAM `Tracker` and writes it in the same format as `isaac_ros_5/launch/extract_paths.py`, so that PyCuVSLAM and Isaac ROS runs can be compared directly and aligned to the ground truth with `results/ground_truth/RunBatchPipeline.py` / `IcpAlign.py`.

Main elements

Element	Description
`RunWriter(out_dir, T_rig_from_base, base_frame, n_cameras, info)`	Collects the output; `T_rig_from_base` converts the rig poses to the base frame
`RunWriter.add_frame(ts_ns, est, slam_pose, obs_counts, n_landmarks, track_ms, tracker)`	Called after each `Tracker.track()`: odometry pose, online SLAM pose, status, SLAM metrics

Element	Description
`RunWriter.finish(tracker, slam_enabled, n_final_landmarks)`	Writes the files; reads the final SLAM trajectory (`get_all_slam_poses`) and the loop closure poses; returns the summary
`cuvslam_version(cuvslam)`	cuVSLAM version string for `run_info.json`

The files it writes are described in File formats.

compare_runs.py

Compares cuVSLAM runs saved in the Isaac ROS CSV format: Isaac ROS runs (`extract_paths.py`) and PyCuVSLAM runs (`--out-dir`). Each run is a folder with `odometry.csv` and/or `slam_poses.csv`, and optionally `status.csv`, `slam_metrics.csv` and `run_info.json`.

Metrics (for `odometry` and `slam_poses` separately)

Group	Metrics
Per run	poses, duration, path length over all consecutive poses (includes frame-to-frame jitter) and over one pose per second (distance actually walked), start-end distance (drift of a closed loop), z range, speed (median, 95th percentile), frames lost, frames with loop closure, median tracking time
Per pair of runs	poses matched by timestamp (`--tolerance-ms`); position difference without alignment (mean, max); after a rigid alignment (Umeyama, rotation + translation): RMSE, maximum, mean and maximum rotation difference
Against the ground truth	`IcpAlign.py` rigid (`--no-scale`): RMSE, symmetric RMSE, mean error; with scale: scale factor and RMSE

Output (`--out`): `report.md`, `metrics.json`, `pairs.csv`, `icp/` (aligned CSV files and logs of `IcpAlign.py`) and `plots/`:

Plot	Content
`<traj>_top_view.png`	top view of all runs, rigidly aligned to the first run; circle = start, square = end
`<traj>_height.png`	z over time in the base frame of the first pose (not gravity-aligned)
`<traj>_pair_difference.png`	position difference between each pair of runs over time, after rigid alignment
`<traj>_vs_ground_truth.png`	runs aligned to the ground truth by `IcpAlign.py` (rigid), with their RMSE
`tracked_points.png`	tracked points per frame (runs with `status.csv` from PyCuVSLAM)

Usage

```
python3 compare_runs.py ros=<isaac ros folder> bag=<folder> edex=<folder> \
    --gt <ground_truth>/paths3d.csv --out <comparison folder>
```

A run is given as `label=folder`, or as `folder` (the label is then the folder name). The first run is the reference of the top-view and height plots.

Arguments and options

Argument	Value	Default	Description
`runs` (positional)	one or more `RUNS`	**required**	run folders, as label=folder or folder

Argument	Value	Default	Description
`--gt`	text	-	ground-truth paths3d.csv (enables the IcpAlign.py comparison)
`--icp-script`	text	`../results/ground_truth/IcpAlign.py`	IcpAlign.py
`--out`	text	**required**	output folder
`--tolerance-ms`	number	`5.0`	maximum timestamp difference to match poses of two runs
`--no-plots`	flag	off	do not create the plots

db3_to_mcap.sh

Reduces a ROS 2 bag to the topics needed by cuVSLAM and converts it to MCAP with internal zstd compression, using `rosbags-convert`. Then checks that the reduced bag can still be converted to EDEX (`bag2edex.py --dry-run`) and prints the size before and after.

Topics kept (for `cam1` and `cam2`): `color/image_raw`, `color/camera_info`, `infra1/image_rect_raw`, `infra1/camera_info`, `infra2/image_rect_raw`, `infra2/camera_info`, `aligned_depth_to_color/image_raw`, `aligned_depth_to_color/camera_info`, `imu`; and `/tf_static`. Edit `CAMS`, `INCLUDED_TOPICS` and `GLOBAL_TOPICS` at the top of the script to change them.

Details

- `--compress zstd --compress-mode storage` is used: the MCAP chunks are compressed internally, which `bag2edex.py` and Rerun can read. The rosbags default (`file`) would produce a `.mcap.zstd` file, and `message` would compress every message: neither can be read by `bag2edex.py`.
- Bags whose `metadata.yaml` points to a `.db3` with a different name than the file present (e.g. `low_detail_0.db3` instead of `low_detail_0-003.db3`) are opened through a temporary folder with a link from the expected name to the real file; the original bag is not modified.
- The destination must not exist (`rosbags-convert` does not overwrite).

Usage

```
./db3_to_mcap.sh <source bag folder> <destination folder>
./db3_to_mcap.sh ~/dotlumen/Captures/10_Low_feature/low_detail_3
~/dotlumen/Captures/mcap/10_Low_feature/low_detail_3
```

Arguments

Argument	Description
`<source bag folder>`	folder of the bag to reduce (`.db3` or `.mcap`, with `metadata.yaml`)
`<destination folder>`	output bag folder; must not exist

all_db3_to_mcap.sh

Runs `db3_to_mcap.sh` on every capture of a folder, with the same capture rules as `convert_all.py` (`converted/` copies preferred; `vslam*`, `converted`, `edex`, `mcap` and the output folder skipped), and mirrors the folder structure. A failed capture does not stop the others.

Output: one reduced bag per capture, `<capture>.log` next to it, and `conversion_summary.tsv` (capture, source, kind, status, seconds, size before and after). Statuses: `ok`, `error`, `no cameras` (bag without camera topics), `skipped`, `destination exists`.

Usage

```
./all_db3_to_mcap.sh [options] [captures folder]
./all_db3_to_mcap.sh -l ~/dotlumen/Captures                    # only list the captures
./all_db3_to_mcap.sh -k low_detail_3 ~/dotlumen/Captures       # one capture
./all_db3_to_mcap.sh -o /data/mcap -s ~/dotlumen/Captures      # other output folder, skip done ones
```

Arguments and options

Argument	Value	Default	Description
`[captures folder]` (positional)	path	`~/dotlumen/Captures`	folder that contains the captures
`-o`	path	`<captures>/mcap`	output folder
`-l`	flag	off	only list the captures found
`-s`	flag	off	skip captures already converted (destination exists)
`-f`	flag	off	reconvert captures already converted (deletes the destination)
`-k`	pattern	-	convert only captures whose relative path contains the pattern; repeatable
`-h`	flag	-	show the usage

`-s` and `-f` are mutually exclusive. Without either, a capture whose destination exists is reported as `destination exists` and not converted.

File formats

EDEX (bag2edex.py, bag2edex_modes.py, merge_rig_edex.py)

File	Content
`stereo.edex`	JSON list `[header, body]`. The header has `cameras` (intrinsics: `distortion_model` `brown5k` or `pinhole`, `distortion_params`, `focal`, `principal`, `size`; `transform`: 3x4 rig-from-camera in the legacy cuVSLAM axes, y up and z backward) and the optional `imu` section (`measurements`, `transform`, noise parameters, `frequency`). The body has `frame_metadata` and `sequence`
`frame_metadata.jsonl`	one JSON line per frame: `frame_id` and `cams` (`id`, `filename`, `timestamp` in ns) for each camera present; partial frames allowed
`IMU.jsonl`	one JSON line per sample: `AngularVelocityX/Y/Z` [rad/s], `LinearAccelerationX/Y/Z` [m/s^2], `timestamp` [ns], in the IMU frame

Saved run output (run_output.py, --out-dir)

File	Content
`odometry.csv`	one pose per tracked frame (visual or visual-inertial odometry)
`slam_poses.csv`	final SLAM trajectory: `get_all_slam_poses()` at the end of the run, i.e. after the loop closure corrections computed by cuVSLAM (as the last `slam_path` of the Isaac ROS node); with `--slam`
`slam_online.csv`	SLAM pose returned frame by frame during the run, without later corrections; with `--slam`

File	Content
`status.csv`	`timestamp, vo_state` (1 = tracked, 2 = lost, as Isaac ROS), `track_time_ms, landmarks, observations_cam<k>`
`slam_metrics.csv`	per frame: `lc_status`, `pgo_status` and landmark counts of the loop closure, as reported by cuVSLAM; with `--slam`
`loop_closures.csv`	last loop closure poses reported by cuVSLAM (it keeps the last 10), in the SLAM world frame
`run_info.json`	script, input, mode, cameras or streams, parameters, cuVSLAM version, base frame and its source, frames, summary metrics

Pose files use the columns of the Isaac ROS CSV files: `timestamp,x,y,z,qx,qy,qz,qw,roll_deg,pitch_deg,yaw_deg`. Timestamps are in seconds; poses are in meters, of the base frame, relative to its first pose (the first row is the identity, as in the Isaac ROS output).

Other files

File	Written by	Content
`bag_report.json`	`bag2edex.py`	per topic: count, rate, gaps, estimated losses; per camera: reference, frames, matches, RGB-IR offsets, image quality, calibration; dropped stale messages
`sync.csv`	`bag2edex.py`, `merge_rig_edex.py`	per frame: timestamp of each stream and offset from the reference
`rig_calibration.json`	`merge_rig_edex.py`	extrinsic used, rig frame, transform of each camera, predicted stereo pairs, IMU transform
`conversion_summary.json`	`convert_all.py`	per capture: source, status, return code, duration, log
`conversion_summary.tsv`	`all_db3_to_mcap.sh`	per capture: source, status, duration, size before and after
`report.md`, `metrics.json`, `pairs.csv`	`compare_runs.py`	comparison tables and metrics

Conventions and notes

- **Timestamps**: the header stamps of the messages are used by default (`--time receive` in `bag2edex.py` uses the bag receive time). cuVSLAM uses the timestamp of camera 0 for the whole frame and expects the cameras of a frame to be within 1 ms.
- **Axes**: the Python API of cuVSLAM and the rig poses use OpenCV axes (x right, y down, z forward). EDEX files store extrinsics in the legacy cuVSLAM axes (y up, z backward); the scripts convert between the two. Saved poses use the ROS base frame `<cam>_link` (x forward, y left, z up). That frame is not gravity-aligned.
- **Stereo pairs**: cuVSLAM forms stereo pairs automatically from the frustum overlap (more than 50% of a 31x31 grid of pixels visible in the other camera between 2 and 4 m). Multicamera and Inertial modes need at least one pair.
- **IMU**: Inertial mode needs one stereo pair and the IMU; with more than two cameras the IMU is used only for gravity. The default noise parameters are those of the Isaac ROS launch files.
- **Rectified images**: the RealSense IR topics are rectified (`image_rect_raw`); `--rectified` sets `rectified_stereo_camera`, as `rectified_images: True` in the Isaac ROS launch files.
- **Determinism**: the tracking scripts run with `async_sba=False` by default (offline, repeatable); `--async-sba` reproduces the asynchronous bundle adjustment of a real-time run.